Assignment

Service Request Log Analyzer

GenAI Level 1 — Limited AI Use ⚠️
This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the *Generative AI Use in Assessments* policy posted at the top of the Assignments section in Moodle.

Scenario

A small property-management company receives maintenance requests from tenants. Each request contains a request ID, category, priority, and estimated repair cost.

The company needs a **C# console application** that processes service requests, makes decisions based on their values, calculates statistics, and saves a summary report to a text file.

Your task is to develop a **Service Request Log Analyzer** demonstrating appropriate use of **operators, flow-control structures, loops, and File I/O**.

Part 1 — Read and Process Service Requests

The company stores incoming maintenance requests in a text file named:

`ServiceRequests.txt`

The file must contain at least **six service requests**, one request per line, using the following format:

```
SR-101,Plumbing,5,450.00
SR-102,General,2,120.00
SR-103,Electrical,4,375.00
SR-104,Heating,3,280.00
SR-105,Plumbing,1,95.00
SR-106,Electrical,5,530.00
```

Each line represents:

```
Request ID, Category, Priority, Estimated Cost
```

Step 0 — Read the Input File

Use **`StreamReader`** or an appropriate **`File`** method to read the service requests from **`ServiceRequests.txt`**.

For example:

```
using StreamReader reader = new StreamReader("ServiceRequests.txt");

while (reader.Peek() >= 0)
{
    string? line = reader.ReadLine();

    // Process the line
}
```

Step 2 — Separate the Data

Use **`Split()`** to separate each line into its individual fields:

```
string[] fields = line.Split(',');
```

The resulting elements represent:

`fields[0]` → **Request ID**

`fields[1]` → **Category**

`fields[2]` → **Priority**

`fields[3]` → **Estimated Cost**

Convert the numeric values to appropriate C# numeric types before performing calculations.

Step 3 — Process Each Request

For every request read from the file, use the required operators and flow-control structures to:

- determine whether the request is normal or urgent;
- determine which maintenance team should handle it;
- calculate total and average estimated costs;
- perform the other required processing in the assignment.

Then **Part 3** writes the processed results to **`ServiceReport.txt`**, and **Part 4** reads that generated report back and verifies it.

The complete data flow for this assignment is:

ServiceRequests.txt
↓
StreamReader
↓
Parse each request
↓
Operators + Flow Control
↓
Process / Calculate
↓
StreamWriter
↓
ServiceReport.txt
↓
StreamReader
↓
Verify and Display

Part 1 — Create and Process Service Requests

Create a Console Application named:

`ServiceRequestAnalyzer`

Create sample data for at least **six service requests**. Each request should contain:

- Request ID
- Category, such as Plumbing, Electrical, Heating, or General
- Priority from **1** to **5**
- Estimated repair cost

Use different values so that your program can demonstrate meaningful decisions and calculations.

Requirements

Use appropriate **arithmetic, relational, logical, and compound-assignment operators** to:

- calculate the total estimated repair cost;
- calculate the average repair cost;
- identify requests costing more than a value you choose;
- determine whether a request is urgent.

For example, your business rule could consider a request urgent when:

`priority >= 4 && estimatedCost > 200`

Display appropriate messages using **`if`**, **`else if`**, and **`else`**.

Part 2 — Use Flow Control

Process all service requests using an appropriate loop such as **`for`** or **`foreach`**.

Your application must demonstrate:

- **`if / else if / else;`**
- at least one **`switch`**;
- **`for`** or **`foreach`**;
- either **`while`** or **`do-while`**;
- `%` (modulo);
- at least one meaningful use of **`break`** or **`continue`**.

For example, a switch could display information based on the request category:

```
switch (category)
{
    case "Plumbing":
        Console.WriteLine("Assign to plumbing team.");
        break;

    case "Electrical":
        Console.WriteLine("Assign to electrical team.");
        break;

    default:
        Console.WriteLine("Assign to general maintenance.");
        break;
}
```

The operators and flow-control structures should have a **logical purpose in the application** rather than being included as unrelated demonstrations.

Part 3 — Write the Report to a File

Create a text report containing the processed results.

The report should contain information such as:

SERVICE REQUEST REPORT

Request: SR-101
Category: Plumbing
Priority: 5
Estimated Cost: $450.00
Status: Urgent

Request: SR-102
Category: General
Priority: 2
Estimated Cost: $120.00
Status: Normal

Total Requests: 6
Total Estimated Cost: $1,850.00
Average Estimated Cost: $308.33

Use either **`StreamWriter`** or an appropriate **`File`** operation to save the report.

When working directly with a stream, use **`using`** so that the resource is properly disposed.

For example:

```
using (StreamWriter writer = new StreamWriter("ServiceReport.txt"))
{
    // Write report information
}
```

Part 4 — Read and Verify the File

After creating the report:

1. Use **`File.Exists()`** to verify that the file was created.
2. Read the report using **`StreamReader`** or an appropriate **`File`** method.
3. Display the file contents in the console.
4. Display the file's **creation or last-modified time**.
5. Create a backup copy named:

`ServiceReport-Backup.txt`

Use appropriate exception handling for the file operations.

For example:

```
try
{
    // File operations
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"File not found: {ex.Message}");
}
catch (UnauthorizedAccessException ex)
{
    Console.WriteLine($"Access denied: {ex.Message}");
}
```

Part 5 — Build and Test

Build and run the completed application.

Your testing should demonstrate different cases, including:

- normal and urgent requests;
- different request categories;
- different priority and cost values;
- calculations using operators;
- execution of the required loops and decision structures;
- successful creation of the report;
- successful reading of the report;
- creation of the backup file.

The final project folder should contain something similar to:

```
ServiceRequestAnalyzer
│
├── ServiceReport.txt
└── ServiceReport-Backup.txt
```

Important: A program that compiles is not necessarily correct. Verify the calculated values, decision logic, console output, and generated files.

Report and Screenshot Requirements

Submit a short report documenting your practical implementation. Include a **comprehensive set of screenshots** demonstrating:

- the important operators and conditions;
- if and switch logic;
- loops and break/continue;
- calculation results;
- file-writing code;
- using and resource management;
- file-reading and exception-handling code;
- successful console output;
- ServiceReport.txt and its contents;
- the backup file;
- successful build and testing.

Screenshots should demonstrate that the requirements were **implemented and executed**, not simply show source code.

In your report, also briefly explain the difference between **&& and &**, the purpose of **break and continue**, the difference between **File, StreamReader/StreamWriter, and FileStream**, and why using is important when working with streams.

Submission and Presentation

Submit the **complete Visual Studio project and report to Moodle** and respect the **due date specified on Moodle**.

In addition to the Moodle submission, **the assignment must be presented to the professor to receive a grade**. During the presentation, you may be asked to run the program, change a condition or service request, explain a loop or operator, and demonstrate how the file is created and read.

An assignment submitted to Moodle but not presented to the professor will not be graded.

Include the required **GenAI Disclosure Statement** according to the Level 1 policy. Failure to include the required GenAI Disclosure Statement will result in a **grade of zero (0)**.

Rubric

Criterion	Weight
Operators, Calculations & Conditional Logic	20%
Flow Control, Loops & break/continue	25%
File Writing, Reading & Resource Management	25%
Exception Handling, File Verification & Backup	15%
Testing, Report, Screenshots & Presentation	15%
Total	**100%**